

K L Deemed to be University

Department of Computer Science and Engineering | 2026-2027, Odd Semester
Database Systems Engineering and Distributed Backend Development (25CS1302E)

Practical Experiment 3: Flexible Data and the Aggregation Pipeline

Name	Habeeb	Roll No.	2520090054
Course	25CS1302E	CO Mapping	CO2
Session	Practical 3	Date	

Aim

To store BookFlow reviews and technical specifications in a MongoDB collection book_metadata using embedded and referenced patterns, query it with comparison operators and regex, build a 'Top Rated' aggregation pipeline (\$match, \$unwind, \$group), and create multikey and text indexes verified with .explain().

Program

p3.js (run with: mongosh --file p3.js, or paste into mongosh)

```
// Experiment 3: Flexible Data and the Aggregation Pipeline (run in mongosh)
use bookflow_db

db.book_metadata.drop()

// Task 1: Document modelling
// - reviews[] : EMBEDDED pattern (reviews live inside the book document)
// - book_id, member_id : REFERENCED pattern (IDs from the PostgreSQL bookflow_db)
// - specs : POLYMORPHIC (e-books store file data, print books store paper data)
db.book_metadata.insertMany([
  {
    _id: "bm1", book_id: 3, title: "The Martian", published_year: 2014,
    genres: ["Science Fiction", "Space"], format: "Digital (EPUB)", rating: 4.7,
    specs: { file_format: "EPUB", file_size_mb: 2.4, drm: false },
    reviews: [
      { member_id: 1, rating: 5, comments: "Brilliant survival story set in space" },
      { member_id: 2, rating: 4, comments: "Funny and very scientific" }
    ]
  },
  {
    _id: "bm2", book_id: 7, title: "Dune", published_year: 2019,
    genres: ["Science Fiction", "Classic"], format: "Paperback", rating: 4.6,
    specs: { pages: 896, paper_weight_gsm: 70, dimensions_cm: [13.3, 3.8, 20.3] },
    reviews: [
      { member_id: 1, rating: 5, comments: "Epic world building on a desert planet" },
      { member_id: 3, rating: 4, comments: "Slow start but worth it" }
    ]
  },
  {
    _id: "bm3", book_id: 10, title: "Project Hail Mary", published_year: 2021,
    genres: ["Science Fiction", "Space"], format: "Digital (PDF) + Audiobook", rating: 4.8,
    specs: { file_format: "PDF", file_size_mb: 5.1, audio_hours: 16.2 },
    reviews: [
      { member_id: 3, rating: 5, comments: "Best space adventure I have read" },
      { member_id: 4, rating: 5, comments: "Rocky is the best character" },
      { member_id: 2, rating: 4, comments: "Great science and humour" }
    ]
  },
  {
    _id: "bm4", book_id: 13, title: "Introduction to Algorithms", published_year: 2022,
    genres: ["Computer Science", "Textbook"], format: "Hardcover", rating: 4.3,
    specs: { pages: 1312, paper_weight_gsm: 80, dimensions_cm: [20.3, 5.6, 23.4] },
    reviews: [
      { member_id: 1, rating: 4, comments: "Dense but complete reference" },
      { member_id: 5, rating: 3, comments: "Too heavy for beginners" }
    ]
  },
  {
    _id: "bm5", book_id: 2, title: "Database System Concepts", published_year: 2019,
    genres: ["Computer Science", "Textbook"], format: "Digital (EPUB)", rating: 3.9,
```

```

    specs: { file_format: "EPUB", file_size_mb: 18.7, drm: true },
    reviews: [
      { member_id: 2, rating: 4, comments: "Clear chapters on normalisation" }
    ]
  },
  {
    _id: "bm6", book_id: 15, title: "Clean Architecture", published_year: 2023,
    genres: ["Computer Science", "Software Design"], format: "Digital (Kindle)", rating: 4.4,
    specs: { file_format: "AZW3", file_size_mb: 6.3, drm: true },
    reviews: [
      { member_id: 4, rating: 5, comments: "Changed how I structure services" },
      { member_id: 5, rating: 4, comments: "Good examples but repetitive" }
    ]
  }
])

// Task 2: Advanced querying
// 2a. Comparison operator: books with a rating greater than 4
db.book_metadata.find(
  { rating: { $gt: 4 } },
  { _id: 0, title: 1, rating: 1 }
).sort({ rating: -1 })

// 2b. $in operator: books in either of two genres
db.book_metadata.find(
  { genres: { $in: ["Space", "Software Design"] } },
  { _id: 0, title: 1, genres: 1 }
)

// 2c. Regex: books whose format includes the word "Digital"
db.book_metadata.find(
  { format: { $regex: /Digital/ } },
  { _id: 0, title: 1, format: 1 }
)

// 2d. Polymorphic data: e-books have file_size_mb, print books have paper_weight_gsm
db.book_metadata.find({ "specs.file_size_mb": { $exists: true } }, { _id: 0, title: 1,
  "specs.file_size_mb": 1 })
db.book_metadata.find({ "specs.paper_weight_gsm": { $exists: true } }, { _id: 0, title: 1,
  "specs.paper_weight_gsm": 1 })

// Task 3: The aggregation pipeline - "Top Rated" report
db.book_metadata.aggregate([
  { $match: { published_year: { $gt: 2020 } } },
  { $unwind: "$reviews" },
  { $group: { _id: "$title",
    avg: { $avg: "$reviews.rating" },
    count: { $sum: 1 } } },
  { $sort: { avg: -1 } },
  { $project: { _id: 0,
    title: "$_id",
    average_rating: { $round: ["$avg", 2] },
    total_reviews: "$count" } }
])

// Task 4: Indexing for speed
// 4a. Without an index, a query on an array field performs a collection scan
db.book_metadata.find({ "reviews.rating": 5 }).explain("queryPlanner").queryPlanner.winningPlan

// 4b. Multikey index on the reviews array, then the same query uses IXSCAN
db.book_metadata.createIndex({ "reviews.rating": 1 })
db.book_metadata.find({ "reviews.rating": 5 }).explain("queryPlanner").queryPlanner.winningPlan

// 4c. Text index on the comments field inside reviews
db.book_metadata.createIndex({ "reviews.comments": "text" }, { name: "reviews_comments_text" })

db.book_metadata.find(
  { $text: { $search: "space" } },
  { _id: 0, title: 1 }
)

// 4d. Verify that the keyword search uses the text index (not a collection scan)
const plan = db.book_metadata.find({ $text: { $search: "space" } }).explain("executionStats")
({
  stage: plan.queryPlanner.winningPlan.stage,
  inputStage: plan.queryPlanner.winningPlan.inputStage.stage,
  indexStage: plan.queryPlanner.winningPlan.inputStage.inputStage.stage,
  indexName: plan.queryPlanner.winningPlan.inputStage.inputStage.indexName,
})

```

```

nReturned:      plan.executionStats.nReturned,
totalKeysExamined: plan.executionStats.totalKeysExamined,
totalDocsExamined: plan.executionStats.totalDocsExamined
})

```

Output

mongosh output

```

test> use bookflow_db
switched to db bookflow_db

bookflow_db> db.book_metadata.drop()
true

// Task 1: insertMany result
{
  acknowledged: true,
  insertedIds: {
    '0': 'bm1',
    '1': 'bm2',
    '2': 'bm3',
    '3': 'bm4',
    '4': 'bm5',
    '5': 'bm6'
  }
}

// 2a. rating > 4 (sorted by rating desc)
[
  { title: 'Project Hail Mary', rating: 4.8 },
  { title: 'The Martian', rating: 4.7 },
  { title: 'Dune', rating: 4.6 },
  { title: 'Clean Architecture', rating: 4.4 },
  { title: 'Introduction to Algorithms', rating: 4.3 }
]

// 2b. genres $in ['Space', 'Software Design']
[
  { title: 'The Martian', genres: [ 'Science Fiction', 'Space' ] },
  { title: 'Project Hail Mary', genres: [ 'Science Fiction', 'Space' ] },
  {
    title: 'Clean Architecture',
    genres: [ 'Computer Science', 'Software Design' ]
  }
]

// 2c. format matches /Digital/
[
  { title: 'The Martian', format: 'Digital (EPUB)' },
  { title: 'Project Hail Mary', format: 'Digital (PDF) + Audiobook' },
  { title: 'Database System Concepts', format: 'Digital (EPUB)' },
  { title: 'Clean Architecture', format: 'Digital (Kindle)' }
]

// 2d. documents that have specs.file_size_mb (e-books)
[
  { title: 'The Martian', specs: { file_size_mb: 2.4 } },
  { title: 'Project Hail Mary', specs: { file_size_mb: 5.1 } },
  { title: 'Database System Concepts', specs: { file_size_mb: 18.7 } },
  { title: 'Clean Architecture', specs: { file_size_mb: 6.3 } }
]

// 2d. documents that have specs.paper_weight_gsm (print books)
[
  { title: 'Dune', specs: { paper_weight_gsm: 70 } },
  {
    title: 'Introduction to Algorithms',
    specs: { paper_weight_gsm: 80 }
  }
]

// 3. Top Rated report (aggregation pipeline)
[
  { title: 'Project Hail Mary', average_rating: 4.67, total_reviews: 3 },
  { title: 'Clean Architecture', average_rating: 4.5, total_reviews: 2 },
  {
    title: 'Introduction to Algorithms',
    average_rating: 3.5,

```

```

    total_reviews: 2
  }
]

// 4a. Before indexing - winning plan for { "reviews.rating": 5 }
{
  stage: 'COLLSCAN',
  filter: { 'reviews.rating': { '$eq': 5 } },
  direction: 'forward'
}

// 4b. createIndex({ "reviews.rating": 1 })
reviews.rating_1

// 4b. After indexing - winning plan for { "reviews.rating": 5 }
{
  stage: 'FETCH',
  inputStage: {
    stage: 'IXSCAN',
    keyPattern: { 'reviews.rating': 1 },
    indexName: 'reviews.rating_1',
    isMultiKey: true,
    multiKeyPaths: { 'reviews.rating': [ 'reviews' ] },
    direction: 'forward',
    indexBounds: { 'reviews.rating': [ '[5, 5]' ] }
  }
}

// 4c. createIndex({ "reviews.comments": "text" })
reviews_comments_text

// 4c. $text search 'space'
[ { title: 'The Martian' }, { title: 'Project Hail Mary' } ]

// 4d. explain("executionStats") summary for the $text query
{
  stage: 'TEXT_MATCH',
  inputStage: 'FETCH',
  indexStage: 'IXSCAN',
  indexName: 'reviews_comments_text',
  nReturned: 2,
  totalKeysExamined: 2,
  totalDocsExamined: 2
}

```

Result

Six polymorphic book documents were stored with embedded reviews and references to SQL book_id/member_id values. \$gt returned five books rated above 4, \$in and the /Digital/ regex returned the matching formats, and \$exists separated e-book specs from print specs. The pipeline ranked post-2020 books by average rating (Project Hail Mary 4.67). The query on reviews.rating changed from COLLSCAN to IXSCAN on a multikey index, and the text search for 'space' used TEXT_MATCH over the reviews_comments_text index instead of a collection scan.